Prepared for self-study · August 2026

Table of Contents

1. Why System Design Matters at This Stage

1.1 What interviewers expect from 3–5 YOE candidates

1.2 How this guide is organized

2. Core Quality Attributes

2.1 Scalability

2.2 Availability

2.3 Reliability & Fault Tolerance

2.4 Latency vs Throughput

2.5 Consistency

3. Networking Fundamentals

3.1 DNS

3.2 TCP vs UDP

3.3 HTTP/HTTPS & TLS

3.4 CDN

4. API Design

4.1 REST principles

4.2 REST vs gRPC vs GraphQL

4.3 Versioning, Pagination, Idempotency

4.4 Rate Limiting

5. Load Balancing

5.1 L4 vs L7 load balancing

5.2 Algorithms

5.3 Health checks & sticky sessions

6. Caching

6.1 Where caching happens

6.2 Caching strategies

6.3 Eviction policies

6.4 Cache invalidation problems

7. Databases

7.1 SQL vs NoSQL

7.2 Indexing

7.3 Normalization vs Denormalization

7.4 ACID vs BASE

7.5 Replication

System Design

A Practical Guide for Java Backend Developers

Target Audience: 3–5 Years Experience

Topic-wise, simple-language explanations of core system design concepts, with Java/Spring-specific notes and interview-oriented tips.

- 7.6 Sharding / Partitioning
- 7.7 Consistent Hashing

8. Messaging & Event-Driven Architecture

- 8.1 Message queues vs pub-sub
- 8.2 Kafka vs RabbitMQ
- 8.3 Event sourcing & CQRS

9. Microservices Architecture

- 9.1 Monolith vs Microservices
- 9.2 Service discovery
- 9.3 API Gateway
- 9.4 Service-to-service communication
- 9.5 Service mesh

10. Distributed Systems Concepts

- 10.1 CAP theorem
- 10.2 Consensus (Raft/Paxos) in simple terms
- 10.3 Distributed transactions: 2PC and Saga
- 10.4 Idempotency & exactly-once thinking

11. Resilience & Fault Tolerance Patterns

- 11.1 Circuit Breaker
- 11.2 Retry & Backoff
- 11.3 Bulkhead
- 11.4 Timeouts
- 11.5 Graceful degradation

12. Security Essentials

- 12.1 AuthN vs AuthZ
- 12.2 OAuth2 & JWT
- 12.3 Encryption basics
- 12.4 Common vulnerabilities

13. Observability

- 13.1 Logging
- 13.2 Metrics
- 13.3 Distributed tracing
- 13.4 Alerting

14. Java & Spring Boot Specific Considerations

- 14.1 Thread pools & concurrency
- 14.2 Connection pooling

14.3 JVM & GC awareness
14.4 Spring Boot production patterns

15. How to Approach a System Design Interview

15.1 The 5-step framework
15.2 Back-of-the-envelope estimation

16. Worked Mini Case Studies

16.1 URL Shortener
16.2 Rate Limiter
16.3 Notification System

1. Why System Design Matters at This Stage

1.1 What interviewers expect from 3–5 YOE candidates

At 3–5 years, you are no longer expected to just write correct code — you are expected to make design decisions that hold up under real traffic, failures, and change. Interviewers at this level are not looking for textbook definitions; they want to see that you can:

- Take a vague requirement ("design an e-commerce cart service") and turn it into a concrete plan.
- Reason about trade-offs out loud — there is rarely one "correct" answer, only good justified choices.
- Estimate scale (users, QPS, data size) and let that estimate drive your design.
- Connect the design back to things you've actually operated: databases, caches, queues, Spring services.

> **Simple way to think about it:** Junior engineers are asked "can you build this?" Mid-level engineers (3–5 YOE) are asked "can you build this so it still works when 10x more people use it, and when a server dies at 2 AM?"

1.2 How this guide is organized

Each topic below explains the concept in plain language first, then gives a short example, and (where relevant) a Java/Spring-specific note showing how the concept shows up in your day-to-day work. Interview tip boxes highlight what to say when the topic comes up in a discussion.

2. Core Quality Attributes

Almost every system design discussion is really a discussion about trade-offs between these five attributes. Learn to name them explicitly — it instantly makes your answers sound structured.

2.1 Scalability

The ability of a system to handle increased load (more users, more data, more requests) without a drop in performance, either by adding more powerful hardware (vertical) or more machines (horizontal).

Vertical scaling

Adding more CPU/RAM to a single machine. Simple, but has a hard ceiling and creates a single point of failure.

Horizontal scaling

Adding more machines and distributing load across them. This is the default approach for modern backend systems because it has no real ceiling, but it requires your application to be **stateless** (see 14.4).

> **Java angle:** A Spring Boot service that stores session data in memory (e.g. HttpSession) cannot be horizontally scaled safely unless you externalize session state to Redis or make the service fully stateless with token-based auth (JWT).

2.2 Availability

The percentage of time a system is up and able to serve requests, usually expressed in "nines" (99.9%, 99.99%...). Achieved through redundancy — no single machine, process, or data center should be a single point of failure.

Availability	Downtime / year	Downtime / month
99% (two nines)	~3.65 days	~7.2 hours
99.9% (three nines)	~8.7 hours	~43 minutes
99.99% (four nines)	~52 minutes	~4.3 minutes
99.999% (five nines)	~5.3 minutes	~26 seconds

2.3 Reliability & Fault Tolerance

Reliability is about the system doing the *correct* thing consistently, not just being "up." Fault tolerance is the system's ability to keep working correctly even when a component fails (a node crashes, a

network call times out, a disk fills up). See Section 11 for concrete patterns.

2.4 Latency vs Throughput

- **Latency** — time taken to complete one request (e.g. 120ms). Usually reported as p50/p95/p99 percentiles rather than averages, because averages hide slow outliers that hurt real users.
- **Throughput** — number of requests a system can handle per unit time (e.g. 5,000 requests/sec).

> **Simple analogy:** Latency is how long it takes one car to cross a bridge. Throughput is how many cars can cross the bridge per minute. You can increase throughput (more lanes) without improving latency for a single car, and vice versa.

2.5 Consistency

Whether all clients see the same data at the same time. In distributed systems this is rarely "free" — you often trade some consistency for availability or performance (see CAP theorem, Section 10.1).

- **Strong consistency** — every read gets the latest write, immediately.
- **Eventual consistency** — reads may return stale data briefly, but all replicas converge to the same value eventually. Common in caches, search indexes, and NoSQL systems like DynamoDB/Cassandra.

> **Interview tip:** When asked "should this be strongly or eventually consistent?", answer with the business impact: "A bank balance needs strong consistency because showing a stale number could let someone overdraw. A 'likes' counter on a social post can be eventually consistent because a 1-2 second delay doesn't hurt anyone."

3. Networking Fundamentals

3.1 DNS (Domain Name System)

Translates a human-readable domain (api.example.com) into an IP address. Think of it as the internet's phone book. DNS resolution is often the first hop in any request and can be a source of latency if not cached properly (TTL settings matter).

3.2 TCP vs UDP

TCP	UDP
Connection-oriented, reliable, ordered delivery	Connectionless, no delivery guarantee
Used by HTTP, databases, most backend APIs	Used by video streaming, DNS queries, gaming
Slower due to handshake + acknowledgements	Faster, lower overhead

3.3 HTTP/HTTPS & TLS

HTTP is the request-response protocol backend services speak. HTTPS wraps HTTP in TLS encryption so data in transit can't be read or tampered with. As a backend developer you rarely implement TLS yourself — it's typically terminated at the load balancer — but you should know terms like "TLS termination" and "mutual TLS (mTLS)" used for service-to-service auth in microservices.

3.4 CDN (Content Delivery Network)

A network of geographically distributed servers that cache static (and sometimes dynamic) content closer to users, reducing latency and offloading traffic from your origin servers. Used for images, JS/CSS bundles, videos, and increasingly for API response caching at the edge.

4. API Design

4.1 REST principles

REST models the system as resources (nouns), manipulated via standard HTTP verbs:

Verb	Meaning	Example
GET	Read a resource	GET /orders/123
POST	Create a resource	POST /orders
PUT	Replace a resource entirely	PUT /orders/123
PATCH	Partially update a resource	PATCH /orders/123
DELETE	Remove a resource	DELETE /orders/123

4.2 REST vs gRPC vs GraphQL

Style	Best for	Notes
REST	Public APIs, simplicity	Human-readable JSON, wide tooling support
gRPC	Internal service-to-service calls	Binary (Protobuf), fast, strongly typed, supports streaming
GraphQL	Clients needing flexible queries (mobile/web BFF)	Client asks for exactly the fields it needs, avoids over/under-fetching

> **Java angle:** Spring Boot supports REST out of the box (@RestController). gRPC in Java typically uses the grpc-java library with generated stubs from .proto files. GraphQL uses graphql-java or the Netflix DGS framework.

4.3 Versioning, Pagination, Idempotency

Versioning

Never break existing clients. Common approaches: URI versioning (/v1/orders), header versioning (Accept: application/vnd.company.v2+json), or query-param versioning.

Pagination

Offset-based (?page=2&size=20) is simple but slow/inconsistent on large, changing datasets. **Cursor-based** (?after=xyz123) is more efficient and stable for large or real-time-updating datasets.

Idempotency

An operation is idempotent if calling it multiple times has the same effect as calling it once. GET, PUT, DELETE should be idempotent by design. POST is not, by default — which is why payment/order APIs use an `Idempotency-Key` header so a retried request (e.g. after a timeout) doesn't create a duplicate charge.

4.4 Rate Limiting

Protects your service from being overwhelmed by too many requests from one client, whether malicious or accidental. Common algorithms:

- **Fixed window** — simple counter reset every N seconds; can allow bursts at window edges.
- **Sliding window** — smooths out the edge-burst problem.
- **Token bucket** — tokens refill at a fixed rate, each request consumes a token; allows controlled bursts. Most widely used in practice.
- **Leaky bucket** — requests processed at a constant fixed rate regardless of burst size.

> **Java angle:** Resilience4j's RateLimiter module, or Bucket4j (built specifically around the token-bucket algorithm), are the common Spring Boot choices.

5. Load Balancing

5.1 L4 vs L7 load balancing

- **Layer 4 (transport layer)** — routes based on IP/port only, doesn't look at request content. Fast, but "dumb."
- **Layer 7 (application layer)** — understands HTTP, can route based on URL path, headers, cookies. Used for things like routing /api/payments to a different backend than /api/search.

5.2 Algorithms

Algorithm	How it works
Round robin	Requests distributed sequentially across servers
Least connections	Sends request to the server with the fewest active connections
Weighted round robin	Servers with more capacity get proportionally more requests
IP hash	Same client IP always routed to same server (useful for sticky sessions)

5.3 Health checks & sticky sessions

Load balancers periodically ping (e.g. GET /health) each backend instance; unhealthy ones are pulled out of rotation automatically. **Sticky sessions** route the same client always to the same backend instance — useful for stateful legacy apps, but works against horizontal scalability, so prefer stateless services where possible.

6. Caching

Caching is one of the highest-leverage tools in system design — a well-placed cache can turn a struggling database into a non-issue.

6.1 Where caching happens

- **Client-side** — browser cache, mobile app local storage.
- **CDN / edge** — caches static assets and some API responses close to the user.
- **Application-level (in-memory)** — e.g. Caffeine cache inside a Java process; fastest, but not shared across instances.
- **Distributed cache** — Redis or Memcached, shared across all instances of a service.
- **Database-level** — query result caching, buffer pools.

6.2 Caching strategies

Strategy	How it works	Trade-off
Cache-aside (lazy loading)	App checks cache first; on miss, reads DB and populates cache	Simple, most common; first request after expiry is slow
Write-through	Every write goes to cache and DB together	Cache always fresh; slightly slower writes
Write-behind (write-back)	Write goes to cache immediately, DB updated asynchronously	Very fast writes; risk of data loss if cache crashes before flush
Read-through	Cache itself is responsible for loading from DB on miss	Cleaner app code, needs cache-provider support

6.3 Eviction policies

Since cache memory is limited, old/unused entries must be evicted: **LRU** (Least Recently Used), **LFU** (Least Frequently Used), and **TTL-based** expiry are the most common.

6.4 Cache invalidation problems

> "There are only two hard things in Computer Science: cache invalidation and naming things." — a well-known engineering joke, and painfully true in practice.

Watch for: **stale data** (cache not updated after a DB write), **cache stampede** (many requests hit an expired key simultaneously and all fall through to the DB at once — mitigate with locks or staggered TTLs), and **thundering herd** on cache-node restart.

> **Java angle:** Spring's `@Cacheable`, `@CachePut`, and `@CacheEvict` annotations with a Redis backend (Spring Data Redis) are the standard combo in a Spring Boot service.

7. Databases

7.1 SQL vs NoSQL

	SQL (RDBMS)	NoSQL
Schema	Fixed, strongly typed	Flexible / schema-less
Relationships	Joins, foreign keys	Denormalized, embedded documents
Consistency	Strong (ACID) by default	Often eventual (BASE), tunable
Scaling	Primarily vertical (horizontal is harder)	Built for horizontal scaling
Examples	MySQL, PostgreSQL, Oracle	MongoDB (document), Cassandra (wide-column), Redis (key-value), Neo4j (graph)
Use when	Transactions, complex queries, strong consistency needed	Massive scale, flexible schema, high write throughput

7.2 Indexing

An index is a data structure (usually a B-Tree) that lets the database find rows without scanning the whole table — similar to a book's index letting you jump to a page instead of reading cover to cover.

- Speeds up reads (WHERE, JOIN, ORDER BY columns) significantly.
- Slows down writes slightly (every INSERT/UPDATE must also update the index).
- Composite indexes matter in column order — an index on (last_name, first_name) helps queries filtering by last_name, but not ones filtering by first_name alone.

7.3 Normalization vs Denormalization

Normalization splits data into related tables to avoid duplication (good for write-heavy, consistency-critical systems). **Denormalization** duplicates data to avoid expensive joins (good for read-heavy systems where speed matters more than storage/duplication). Most real systems use a mix.

7.4 ACID vs BASE

ACID (traditional RDBMS)	BASE (many NoSQL systems)
Atomicity — all or nothing	**B**asically **A**vailable
Consistency — valid state always	**S**oft state — state may change over time
Isolation — concurrent txns don't interfere	**E**ventual consistency
Durability — committed data survives crashes	

7.5 Replication

Copying data across multiple database nodes for availability and read scaling.

- **Leader-follower (master-replica)** — writes go to the leader, reads can be served from followers. Simplest and most common.
- **Multi-leader** — multiple nodes accept writes; more complex conflict resolution.
- **Leaderless** (e.g. Cassandra/DynamoDB style) — any node accepts reads/writes, uses quorum reads/writes for consistency.

7.6 Sharding / Partitioning

Splitting a large dataset across multiple database instances so no single machine holds all the data.

- **Range-based sharding** — e.g. users A-M on shard 1, N-Z on shard 2. Simple but can create hot spots.
- **Hash-based sharding** — hash(user_id) % N decides the shard. Spreads load evenly but makes range queries harder.
- **Directory-based sharding** — a lookup service maps keys to shards; flexible but adds a dependency.

7.7 Consistent Hashing

A technique that minimizes data movement when nodes are added or removed from a distributed cache/database. Instead of plain `hash(key) % N` (which reshuffles almost everything when N changes), consistent hashing places both nodes and keys on a virtual ring; only a small fraction of keys move when a node joins or leaves. Used by Cassandra, DynamoDB, and many distributed caches.

> **Interview tip:** If asked "how would you scale this database?", walk through the ladder: add an index → add a read replica → add caching → then shard, in that order. Jumping straight to sharding without justifying the earlier steps is a common red flag.

8. Messaging & Event-Driven Architecture

8.1 Message queues vs pub-sub

- **Queue (point-to-point)** — each message is consumed by exactly one consumer. Good for distributing work among a pool of workers.
- **Publish-subscribe** — each message is delivered to every subscriber. Good for broadcasting events to multiple independent services.

Benefits: decouples producer and consumer (they don't need to be online at the same time), smooths out traffic spikes (buffering), and enables asynchronous processing so the user-facing request doesn't wait on slow work.

8.2 Kafka vs RabbitMQ

	Kafka	RabbitMQ
Model	Distributed log (partitioned topics)	Traditional message broker (queues/exchanges)
Throughput	Very high, built for streaming	Moderate, optimized for complex routing
Message retention	Retained for a configurable time even after consumption	Typically removed once acknowledged
Ordering	Guaranteed within a partition	Guaranteed within a queue
Use when	Event streaming, log aggregation, high-volume pipelines	Task queues, complex routing rules, RPC-style messaging

> **Java angle:** Spring Kafka and Spring AMQP (for RabbitMQ) are the standard integration libraries. Know the difference between `@KafkaListener` consumer groups and manual vs auto offset commits — a very common interview follow-up.

8.3 Event sourcing & CQRS

Event sourcing stores every state change as an immutable event (rather than just the current state), so the current state is derived by replaying events. Gives a full audit trail and time-travel debugging, at the cost of complexity.

CQRS (Command Query Responsibility Segregation) separates the write model (commands that change state) from the read model (optimized queries), often backed by different data stores. Frequently paired with event sourcing, but can be used independently.

9. Microservices Architecture

9.1 Monolith vs Microservices

	Monolith	Microservices
Deployment	One unit	Independently deployable services
Scaling	Scale the whole app together	Scale individual services based on their load
Complexity	Simple to start, harder to grow	Operationally complex, but teams scale independently
Failure isolation	One bug can bring down everything	A failing service can be isolated (with the right patterns)

> **Simple rule of thumb:** Don't default to microservices for a new product. Start with a well-structured monolith; split into services once you have real evidence of scaling or team-boundary pain. Premature microservices are a very common real-world mistake.

9.2 Service discovery

In a dynamic environment (containers scaling up/down, IPs changing), services need a way to find each other. **Client-side discovery** (e.g. Eureka + Ribbon/Spring Cloud LoadBalancer) has the caller query a registry directly. **Server-side discovery** (e.g. Kubernetes Services) has a proxy/load balancer handle the lookup transparently.

9.3 API Gateway

A single entry point that sits in front of all microservices, handling cross-cutting concerns: routing, authentication, rate limiting, request/response transformation, and aggregation. Examples: Spring Cloud Gateway, Kong, AWS API Gateway.

9.4 Service-to-service communication

- **Synchronous** (REST/gRPC) — simple to reason about, but couples services' availability together (if B is down, A's request fails).
- **Asynchronous** (message queue/event) — more resilient and decoupled, but harder to trace and debug.

9.5 Service mesh

An infrastructure layer (e.g. Istio, Linkerd) that handles service-to-service communication concerns — retries, mTLS, load balancing, observability — via sidecar proxies, without the application code

needing to implement them. Mostly relevant once you're running dozens of microservices on Kubernetes.

10. Distributed Systems Concepts

10.1 CAP theorem

In a distributed system, when a network partition happens, you must choose between:

- **Consistency** — every node sees the same data.
- **Availability** — every request gets a response (even if not the latest data).

You cannot have both during a partition (Partition tolerance, "P", is not optional in a real distributed system, so the real choice is CP vs AP).

> **Simple way to remember it:** If the network between two data centers breaks, do you (a) refuse to answer until you're sure the data is correct (choosing **C**onsistency), or (b) answer anyway with possibly-stale data (choosing **A**vailability)? You must pick one.

Examples: MongoDB and HBase lean CP; Cassandra and DynamoDB lean AP (tunable per-query).

10.2 Consensus (Raft/Paxos) in simple terms

Consensus algorithms let a cluster of nodes agree on a single value (e.g. "who is the leader?" or "what is the next entry in the log?") even if some nodes fail or messages are delayed. You will rarely implement Raft/Paxos yourself — but tools you use every day rely on them: **ZooKeeper** and **etcd** use variants of these algorithms for leader election and configuration consistency (used internally by Kafka, Kubernetes, etc.).

10.3 Distributed transactions: 2PC and Saga

Two-Phase Commit (2PC)

A coordinator asks all participants to "prepare" a transaction; only if everyone agrees does it tell them all to "commit." Guarantees atomicity across services but is blocking — if the coordinator or a participant crashes mid-way, resources can be locked for a long time. Rarely used across microservices in practice.

Saga pattern

Breaks a distributed transaction into a sequence of local transactions, each with a compensating action to undo it if a later step fails. E.g. "Reserve inventory → Charge payment → Ship order" — if payment fails, a compensating "release inventory" step runs. Two flavors: **choreography** (services react to each other's events, no central coordinator) and **orchestration** (a central saga orchestrator tells each service what to do next).

> **Interview tip:** Sagas are the realistic, expected answer for "how do you keep data consistent across microservices" — 2PC is mentioned mainly to show you understand why it's avoided at scale.

10.4 Idempotency & exactly-once thinking

Networks fail and retries happen, so most distributed systems can only realistically guarantee **at-least-once** delivery. "Exactly-once" processing is usually achieved not by magic, but by combining at-least-once delivery with an idempotent consumer (e.g. checking a processed-message ID table before acting on a Kafka message).

11. Resilience & Fault Tolerance Patterns

11.1 Circuit Breaker

Prevents a service from repeatedly calling a downstream dependency that's already failing, which would waste resources and worsen the outage. States: **Closed** (calls flow normally) → **Open** (after too many failures, calls fail fast without even trying) → **Half-Open** (after a cooldown, a few test calls check if the dependency has recovered).

> **Java angle:** Resilience4j's `@CircuitBreaker` annotation is the modern standard in Spring Boot (Netflix Hystrix is now in maintenance mode / deprecated).

11.2 Retry & Backoff

Automatically retry a failed call, but with **exponential backoff** (wait longer between each retry) and **jitter** (random variation) to avoid many clients retrying in lockstep and overwhelming the recovering service (the "thundering herd" problem).

11.3 Bulkhead

Named after ship compartments that stop one flooded section from sinking the whole ship. Isolates resources (e.g. separate thread pools per downstream dependency) so a slow/failing dependency can't exhaust the threads needed to serve requests to a healthy dependency.

11.4 Timeouts

Every network call must have a timeout. Without one, a single slow dependency can tie up threads/connections indefinitely and cascade into a full outage. Set timeouts deliberately — too short causes false failures, too long delays failure detection.

11.5 Graceful degradation

When a non-critical dependency fails, serve a reduced but still useful experience instead of failing the whole request — e.g. if a "recommended products" service is down, show the product page without recommendations rather than a 500 error.

12. Security Essentials

12.1 AuthN vs AuthZ

- **Authentication (AuthN)** — verifying who you are (login).
- **Authorization (AuthZ)** — verifying what you're allowed to do (permissions/roles).

12.2 OAuth2 & JWT

OAuth2 is a delegation framework that lets a user grant a third-party app limited access to their resources without sharing their password (e.g. "Sign in with Google"). **JWT (JSON Web Token)** is a compact, signed token format often used as the access token in OAuth2 flows and for stateless session management — the server can verify the token's signature without a database lookup, which is exactly what enables stateless, horizontally scalable services.

> **Java angle:** Spring Security with the OAuth2 Resource Server starter is the standard way to validate incoming JWTs in a Spring Boot service.

12.3 Encryption basics

- **Encryption in transit** — TLS on all network calls, including internal service-to-service traffic where feasible (mTLS).
- **Encryption at rest** — database/disk-level encryption for stored data.
- **Symmetric vs asymmetric** — symmetric (same key encrypts/decrypts, fast, e.g. AES) vs asymmetric (public/private key pair, used for things like TLS handshakes and JWT signing with RS256).

12.4 Common vulnerabilities

Be able to name and explain, in one line each: **SQL injection** (untrusted input concatenated into a query — prevent with parameterized queries/prepared statements), **XSS** (untrusted input rendered as executable script in a browser), **CSRF** (a malicious site tricks a logged-in user's browser into making unwanted requests), and the value of rate limiting/input validation at every trust boundary.

13. Observability

You cannot operate a distributed system you cannot see into. Observability is often summarized as three pillars:

13.1 Logging

Structured (JSON) logs with consistent fields (timestamp, service name, trace ID, log level) so they can be searched and correlated centrally (e.g. ELK stack — Elasticsearch, Logstash, Kibana — or a cloud equivalent).

13.2 Metrics

Numeric time-series data (request count, error rate, latency percentiles, JVM heap usage) collected at regular intervals and visualized on dashboards. Prometheus + Grafana is the most common open-source combo; Spring Boot Actuator + Micrometer is the standard way to expose metrics from a Java service.

13.3 Distributed tracing

Tracks a single request as it flows across multiple microservices, showing where time is spent at each hop. Essential for debugging latency issues in a microservices architecture. A **trace ID** is generated at the entry point and propagated through every downstream call. Tools: Zipkin, Jaeger, or OpenTelemetry (the emerging vendor-neutral standard).

13.4 Alerting

Automated notifications (PagerDuty, Slack, email) triggered when a metric crosses a threshold (e.g. error rate > 5% for 5 minutes). Good alerts are actionable and tied to user-facing impact (SLOs), not noisy internal fluctuations.

14. Java & Spring Boot Specific Considerations

System design interviews for Java backend roles often circle back to "how does this actually run in your stack?" This section connects the abstract concepts above to the JVM/Spring world.

14.1 Thread pools & concurrency

A Spring Boot app handling HTTP requests uses a thread pool (Tomcat's default, or a reactive event loop with WebFlux). Each blocking call (DB query, downstream HTTP call) ties up a thread until it returns. Under load, an undersized thread pool or a slow downstream call can exhaust all threads and stall the whole service — this is exactly why bulkheads and timeouts (Section 11) matter in practice, not just in theory.

> **Blocking vs reactive:** Traditional Spring MVC is thread-per-request (blocking). Spring WebFlux uses a small number of event-loop threads and non-blocking I/O, which scales better under high concurrency with slow I/O-bound calls, at the cost of a steeper learning curve (reactive streams, Mono/Flux).

14.2 Connection pooling

Opening a new database connection per request is expensive. Connection pools (HikariCP is the Spring Boot default) keep a set of ready-to-use connections open and hand them out/return them per request. Pool size must be tuned relative to your DB's max connections and your service's expected concurrency — an oversized pool across many service instances can actually overwhelm the database.

14.3 JVM & GC awareness

You don't need to be a GC-tuning expert, but you should know: the JVM heap has generations (young/old), most objects die young (Java is optimized for this), and long GC pauses show up as latency spikes/p99 outliers in your metrics. Modern collectors (G1GC, ZGC) are designed to minimize pause times for latency-sensitive services.

14.4 Spring Boot production patterns

- **Statelessness** — avoid in-memory session/state so any instance can serve any request, enabling horizontal scaling and safe rolling deployments.
- **Externalized configuration** — Spring Cloud Config or environment variables, not hardcoded values, so the same artifact runs across environments.
- **Health & readiness probes** — Actuator's `/actuator/health` endpoint feeds Kubernetes liveness/readiness checks so unhealthy pods are restarted or removed from rotation automatically.

- **Graceful shutdown** — finish in-flight requests before the container stops, avoiding dropped requests during deployments.

15. How to Approach a System Design Interview

15.1 The 5-step framework

1. **Clarify requirements** — functional (what must it do?) and non-functional (scale, latency, consistency needs). Ask questions instead of assuming.
2. **Estimate scale** — users, requests/sec, data volume/growth (see 15.2).
3. **High-level design** — draw the major components (client, API gateway, services, database, cache, queue) and how they connect, before diving into any one piece.
4. **Deep dive** — the interviewer will usually steer you into 1-2 components to go deeper on (e.g. "how would you shard this database?"). This is where Sections 6-13 above get applied.
5. **Identify bottlenecks & trade-offs** — proactively call out weak points and how you'd address them (single points of failure, hot partitions, cache stampedes, etc.).

15.2 Back-of-the-envelope estimation

A few numbers worth memorizing to sanity-check any design out loud:

Quantity	Rough value
1 million requests/day	~12 requests/sec average
Read:Write ratio for typical web apps	Often 100:1 (reads dominate → cache aggressively)
1 KB × 1 million rows	~1 GB of data
Redis / in-memory cache read	~sub-millisecond
SQL query over a network	~1–10 ms (indexed), much higher without an index at scale
Cross-region network call	~100–150 ms

> **Interview tip:** You don't need exact numbers — the goal is to show that peak traffic (not average) drives capacity planning, e.g. "if average is 12 req/sec but traffic spikes 10x during sales, we should design for ~120 req/sec peak."

16. Worked Mini Case Studies

Short, structured walkthroughs applying the framework above. These are intentionally compact — the goal is to show the shape of a good answer, not an exhaustive spec.

16.1 URL Shortener (e.g. bit.ly)

Requirements

Shorten a long URL to a short code; redirect short code → original URL; 100M new URLs/month, heavily read-dominant (redirects >> creations).

Design

- Generate a short code using a base62-encoded auto-incrementing ID (or a hash + collision check) — avoids needing to check for collisions on every write if IDs are pre-allocated in ranges per app instance.
- Store `short_code → long_url` mapping in a key-value store (fast lookups) or a SQL table with an index on short_code.
- Cache hot short codes in Redis — redirects are read-heavy, so this is the highest-leverage optimization (Section 6).
- Use a CDN/edge redirect for extremely popular links where possible.

Bottlenecks called out

Hot keys (a viral link) — mitigate with caching; ID generation contention across multiple app instances — mitigate by pre-allocating ID ranges per instance instead of a shared counter.

16.2 Rate Limiter (as a shared service)

Requirements

Limit each API client to N requests per minute, enforced consistently across many stateless service instances.

Design

- Use the **token bucket** algorithm (Section 4.4) for its balance of simplicity and burst tolerance.
- Store each client's token count in a shared, low-latency store — **Redis**, using an atomic `INCR` + `EXPIRE` (or a Lua script for atomicity) so multiple service instances don't race each other.
- Enforce the limit at the API Gateway (Section 9.3) so individual services don't each reimplement it.

Bottlenecks called out

Redis itself becomes a single point of failure/bottleneck at very high scale — mitigate with Redis Cluster and/or a local in-memory approximation with periodic sync for extreme-scale systems.

16.3 Notification System

Requirements

Send email/SMS/push notifications triggered by events across the platform (order shipped, password reset, etc.), reliably and without blocking the triggering service.

Design

- Triggering services publish an event to a **message queue/topic** (Kafka) rather than calling a notification API synchronously — decouples and smooths bursts (Section 8).
- A notification service consumes events, determines the channel(s) and template, and calls the relevant third-party provider (SendGrid, Twilio, FCM).
- Wrap each provider call in a **circuit breaker + retry with backoff** (Section 11); failed sends go to a dead-letter queue for inspection/replay instead of being silently dropped.
- Track delivery status per notification in a database for auditability and user-facing "notification history."

Bottlenecks called out

Third-party provider rate limits — mitigate with client-side rate limiting/backpressure before calling out; duplicate sends on consumer retry — mitigate with an idempotency key per notification event (Section 10.4).